

Estruturas de Repetição em Python

Módulo 3 (continuação) — ensinando o programa a repetir tarefas

Objetivo da semana

Usar `while` e `for` para repetir blocos de código, entender contadores, acumuladores e `range()`, iterar sobre strings, e evoluir o raciocínio com laços aninhados.

O que vamos ver hoje

1 O que é um laço (loop)

2 while — contadores e acumuladores

3 for e range()

4 Iterando sobre strings

5 Repetições aninhadas

O que é um laço (loop)

Repetir código sem copiar e colar a mesma linha várias vezes.

- Laço (ou loop): estrutura que repete um bloco de código enquanto uma condição for verdadeira, ou um número definido de vezes
- Sem laços, para repetir uma tarefa 100 vezes teríamos que escrever a mesma linha 100 vezes — os laços resolvem isso
- Python tem dois tipos principais de laço: `while` (repete enquanto uma condição for True) e `for` (repete percorrendo uma sequência)
- Assim como no `if`, o bloco do laço é definido pela indentação, e a linha do laço termina com dois-pontos

```
contador = 1
while contador <= 5:
    print(contador)
    contador += 1
```

while — contadores e acumuladores

O laço condicional: repete enquanto a condição continuar True.

- while repete o bloco enquanto a condição for True — quando ela virar False, o laço para
- Contador: uma variável que aumenta (ou diminui) a cada repetição, geralmente usada para controlar quantas vezes o laço roda
- Acumulador: uma variável que vai somando (ou multiplicando) valores a cada repetição, para chegar num total no final
- Cuidado com o laço infinito: se a condição nunca virar False, o programa nunca para — sempre garanta que algo dentro do laço muda a condição
- O operador += é um atalho: contador += 1 é o mesmo que contador = contador + 1

```
soma = 0
numero = 1
while numero <= 10:
    soma += numero
    numero += 1
print(f"A soma de 1 a 10 é {soma}")
```

for e range()

Quando já sabemos exatamente quantas vezes repetir.

- for percorre uma sequência de valores, executando o bloco uma vez para cada valor
- range(n) gera uma sequência de números de 0 até n-1 — muito usado junto com o for para repetir um número definido de vezes
- range(inicio, fim) gera números de inicio até fim-1; range(inicio, fim, passo) permite pular de tantos em tantos
- O for com range() é normalmente mais direto que o while quando já sabemos quantas repetições precisamos
- A variável do for (muitas vezes chamada de i) recebe um valor novo a cada volta do laço, automaticamente

```
for i in range(1, 6):  
    print(f"Repetição número {i}")
```

Iterando sobre strings

O for também percorre o conteúdo de uma string, caractere por caractere.

- Uma string é uma sequência de caracteres — e o for pode percorrer essa sequência diretamente, sem precisar de range()
- A cada repetição, a variável do for recebe um caractere da string, na ordem em que aparecem
- Isso é muito útil para contar ocorrências de uma letra, verificar caracteres específicos, ou construir uma nova string caractere a caractere
- Combinando for com if, dá pra fazer coisas como contar quantas vezes uma vogal aparece numa palavra

```
palavra = "python"
for letra in palavra:
    print(letra.upper())
```

Repetições aninhadas

Um laço dentro de outro laço, para percorrer combinações.

- Assim como o if, o for e o while também podem ser aninhados — um laço completo dentro do bloco de outro
- A cada volta do laço externo, o laço interno roda por completo — por isso o número total de repetições multiplica
- Clássico exemplo: gerar a tabuada de vários números de uma vez, usando um for externo (o número) e um for interno (o multiplicador)
- Cuidado com a indentação: cada nível de laço tem seu próprio recuo, igual vimos com if aninhado na semana passada



```
for numero in range(1, 4):  
    for multiplicador in range(1, 4):  
        resultado = numero *  
multiplicador  
        print(f"{numero} x  
{multiplicador} = {resultado}")
```

Mini-projeto: Sistema de Notas da Turma

Usando for/range para percorrer os alunos, mais um contador e um acumulador — juntando laços com o if da semana passada.

```
● ● ●

qtd_alunos = int(input("Quantos alunos tem a turma? "))
soma_notas = 0
aprovados = 0

for i in range(qtd_alunos):
    nota = float(input(f"Nota do aluno {i + 1}: "))
    soma_notas += nota
    if nota >= 6:
        aprovados += 1

media_turma = soma_notas / qtd_alunos
print(f"Média da turma: {media_turma:.2f}")
print(f"Aprovados: {aprovados} de {qtd_alunos}")
```

Exercícios de aula

1. Use um for com range() para imprimir os números de 1 a 10.
2. Use um while para imprimir uma contagem regressiva de 10 até 1.
3. Peça um número e use for com range() para imprimir a tabuada desse número, de 1 a 10.
4. Use for para percorrer o nome da pessoa (uma string) e imprimir cada letra em uma linha separada.

Tarefa de casa

1. Peça um número inteiro positivo e use um laço para calcular e imprimir o fatorial desse número (ex.: fatorial de 5 é $5 \times 4 \times 3 \times 2 \times 1 = 120$).
2. Simule uma verificação de senha: defina uma senha fixa no código e use um while para pedir a senha repetidamente até o usuário acertar, contando e exibindo no final quantas tentativas foram usadas.

DESAFIO INTERMEDIÁRIO

Construa um verificador de números primos: peça ao usuário um número inteiro maior que 1. Usando um laço for, teste se algum número entre 2 e o número menos 1 divide ele exatamente (resto zero). Um número primo é aquele que não é divisível por nenhum número além de 1 e ele mesmo. Ao final, informe se o número é primo ou não, e quantos divisores diferentes de 1 e dele mesmo foram encontrados.

Avaliação e encerramento

Questionário 5 (online, individual) disponível ao final da semana.

Até a Semana 6!